CSE 111 Fall 2026: Project Checkpoint 1
LinkHyper: Information Cataloguing as Social Media
Ryan Hsiao and Sathya McKinzie

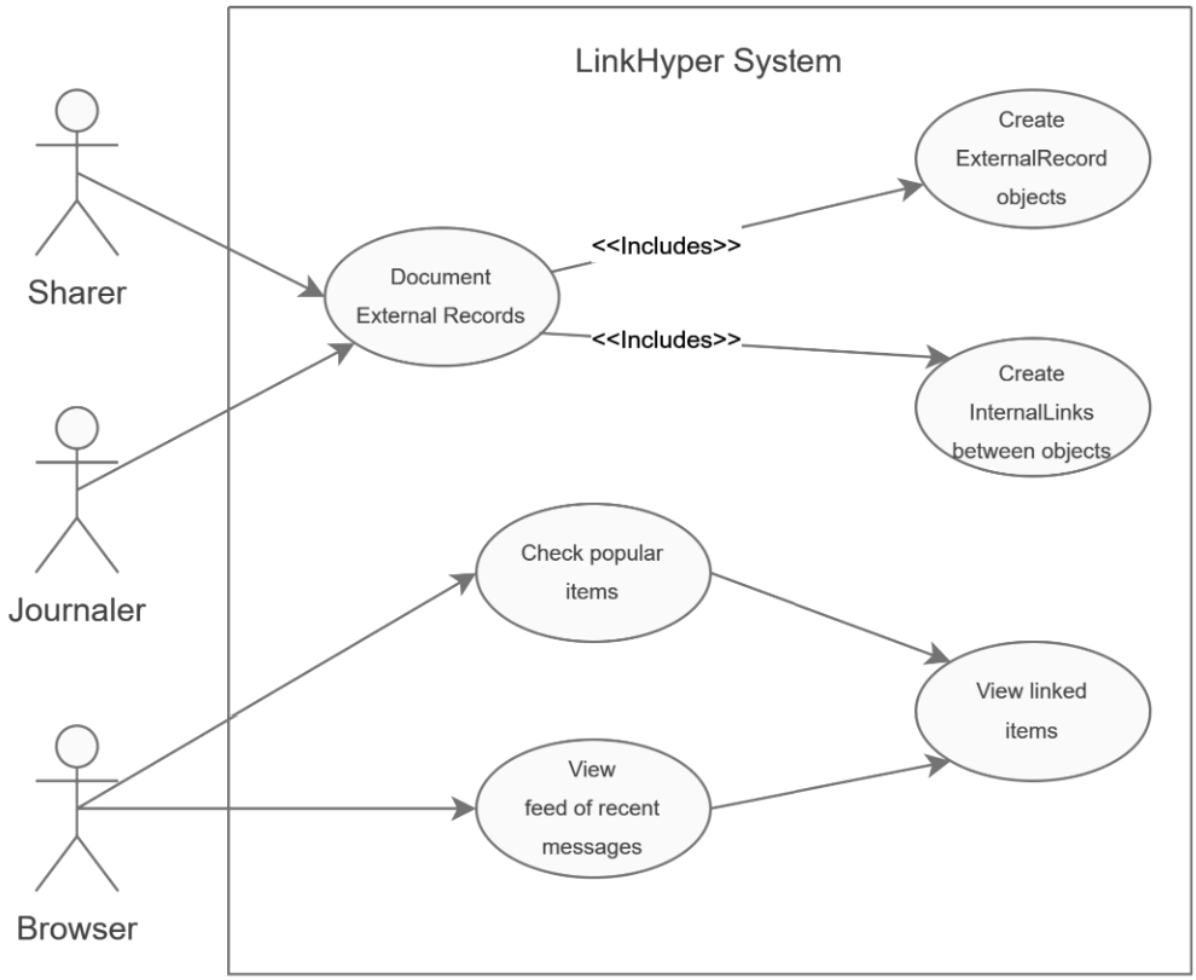
Synopsis

LinkHyper is a forum based web application based around sharing outlinks to external media such as internet articles, books, or music albums. Users can catalog any sort of media by creating a tuple with an external identifier (like a URL, ISBN, or an ID string in a third-party database), the name of the external system from which the identifier is given, and any other applicable metadata. As is typical in forum systems, users can comment on and discuss catalogued items. In addition, users can catalog links between different items, documenting relationships such as one item citing, quoting, interpolating, or referencing another. Content discovery in LinkHyper includes traditional modes such as in chronological feeds and popularity-based leaderboards, but also through the user-defined links between items.

Existing social media platforms already emphasize sharing of external content, but LinkHyper takes a different approach by emphasizing consistent documentation of links to and between items. This not only enables novel mental models and user interaction, but also encourages proper attribution as opposed to rampant intellectual property violation that is the standard when reuploading external content in existing social media platforms. Besides the social features, LinkHyper also enables users to create structured collections of their interests across different types of media. Rather than users having to encounter a “Black Box” algorithm like they would on other sites, our website emphasizes customizability, where users can manually filter and block certain domains, like news sites, and types of media from appearing.

As required in the checkpoint 1 instructions, the GitHub repository used for this project is https://github.com/RyanMHsiao/link-hyper, and the materials for checkpoint 1 (this specs document and the presentation slides) are uploaded to that repository.
Authors & Contribution: Ryan Hsiao 40%, Sathya McKinzie 60%

Use Case Diagram

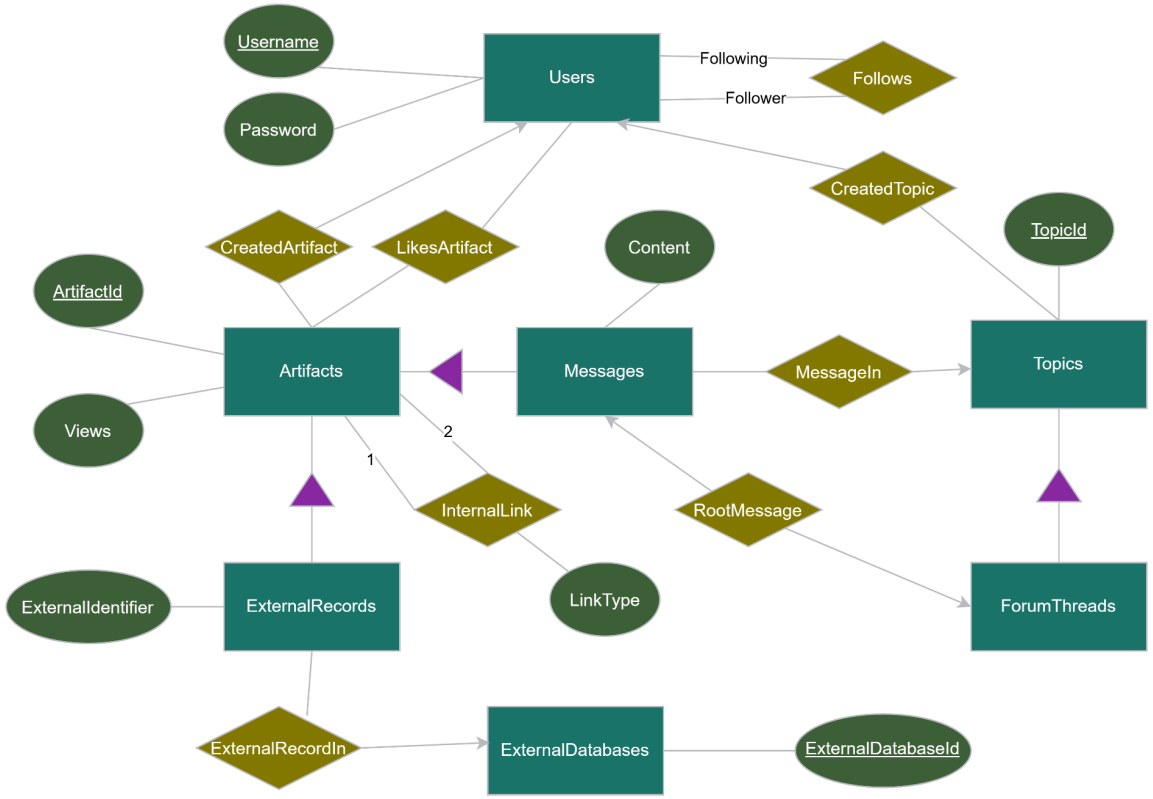


Since this is a service targeted at consumers and not meant for business applications, the different types of users are based on interests rather than strictly defined roles. The sharer wants to show other users certain interesting items, analogous to content reposters in existing social media. The journaler is focused mostly on recording a personal catalogue of items in a structured format, analogous to people who use specialized software for writing and organizing notes or keeping track of consumed pieces of media. The browser wants to see what interesting items other people have shared, analogous to people who go through review sites and online databases to explore their interests.

Authors & Contribution: Ryan Hsiao 30%, Sathya McKinzie 70%

ER Diagram

The database consists of 7 entity sets (Users, ForumThreads, Topics, Messages, ExternalRecords, Artifacts, ExternalDatabases) connected via the 11 following relationships: ForumThreads isa Topics, Messages isa Artifacts, ExternalRecords isa Artifacts, CreatedTopic (1:N between Users and Topics), CreatedArtifact (1:N between Users and Artifacts), RootMessage (1:1 between ForumThreads and Messages), MessageIn (1:N between Topics and Messages), ExternalRecordIn (1:1 between ExternalRecords and ExternalDatabases), UserFollowing (M:N between Users and Users), LikesArtifact (M:N between Users and Artifacts), and InternalLink (M:N between Artifacts and Artifacts)



Authors & Contribution: Ryan Hsiao 70%, Sathya McKinzie 30%

Relational Schema

Each entity set and many to many relationship is given its own SQL relation, meaning subclasses are handled in E/R style and resulting in a total of 10 relations. There are no weak entity sets, so there is no ambiguity there as to how the conversion took place. The number of attributes is intentionally kept to a minimum for now for the sake of simplicity, and more may be added as they are needed.

The names of the relations representing entity sets are changed from plural to singular in order to better match SQL conventions. The attribute names for the keys are chosen to emphasize clarity of the corresponding attribute in the ER diagram. The length of strings and decimals are chosen arbitrarily and may b

User

```
CREATE TABLE user (
 username char(20) not null,
 password char(20) not null,
 PRIMARY KEY (username)
);
```

Authors & Contribution: Ryan Hsiao 60%, Sathya McKinzie 40%

Artifact

Artifact is the chosen technical term used for items that can be linked in this project. Since subclasses are not mutually exclusive in ER diagrams, note that it is possible for an artifact to be both a message and an external record, though for most cases an artifact will be one or another.

```
CREATE TABLE artifact (
  artifact_id decimal(9, 0) not null,
  views integer not null,
  username char(20) not null, -- CreatedArtifact
  PRIMARY KEY (artifact_id)
);
```

Authors & Contribution: Ryan Hsiao 60%, Sathya McKinzie 40%

Message

This is the relation containing what would normally be called posts or comments in other social media apps.

```
CREATE TABLE message (
  artifact_id decimal(9, 0) not null,
  topic_id decimal(9, 0) not null, -- MessageIn
  -- char and varchar have very limited length
  content text not null
);
```

Authors & Contribution: Ryan Hsiao 60%, Sathya McKinzie 40%

External Record

This represents an item that is stored and uniquely identified in an external database. The most general case is for the external identifier to consist of a URL. A more specific example of an external identifier is an ISBN for a book.

```
CREATE TABLE externalrecord (
  artifact_id decimal(9, 0) not null,
  external_database_id decimal(9, 0) not null,
  external_identifier varchar(200) not null
);
```

Authors & Contribution: Ryan Hsiao 60%, Sathya McKinzie 40%

Topic

A general collection of items, with the current only subclass being a thread of messages as would be seen in a forum.

```
CREATE TABLE topic (
  topic_id decimal(9, 0) not null,
  username char(20) not null, -- CreatedTopic
  PRIMARY KEY (topic_id)
);
```

Authors & Contribution: Ryan Hsiao 60%, Sathya McKinzie 40%

Forum Thread

```
CREATE TABLE forumthread (
  topic_id decimal(9, 0) not null,
  artifact_id decimal(9, 0) not null -- RootMessage
);
```

Authors & Contribution: Ryan Hsiao 60%, Sathya McKinzie 40%

External Database

```
CREATE TABLE externaldatabase (
  external_database_id decimal(9, 0) not null,
  PRIMARY KEY(external_database_id)
);
```

Authors & Contribution: Ryan Hsiao 60%, Sathya McKinzie 40%

Follows

This relation and the two relations afterward are all used to represent many-to-many relationships from the ER diagram.

```
CREATE TABLE follows (
 follower char(20) not null,
 following char(20) not null
);
```

Authors & Contribution: Ryan Hsiao 60%, Sathya McKinzie 40%

Likes Artifact

```
CREATE TABLE likesartifact (
 username char(20) not null,
 artifact_id decimal(9, 0) not null
);
```

Authors & Contribution: Ryan Hsiao 60%, Sathya McKinzie 40%

Internal Link

Links can be created between different artifacts. Some link types are predefined and have a standard meaning, such as marking that artifact 1 is an adaptation of artifact 2. Users are also able to define their own link types, which do not have a predefined meaning and can represent any sort of relationship between the two artifacts.

```
CREATE TABLE internallink (
 artifact_1 decimal(9, 0) not null,
 artifact_2 decimal(9, 0) not null,
```

```
 link_type char(20) not null
);
```

Authors & Contribution: Ryan Hsiao 60%, Sathya McKinzie 40%